

Plano de Ação — Projeto Analisador de Currículos

Versionamento, Setup do Repositório e Divisão de Sprints

10/09/2026

Contexto

Time de 6 desenvolvedores em atividade — Kevin Carvalho Iqbal, Gabriel Kalebe, Carlos Eduardo Oliveira, Allan da Silva, Gustavo Souto Pereira e Vitor Bittencourt dos Santos, todos já ativos a partir da Sprint 1. Gustavo entra já na Sprint 1 assumindo o que seria a fatia inicial de Carlos Eduardo (US-004 + apoio de infraestrutura), que passa a atuar a partir da Sprint 2. Vitor também entra já na Sprint 1, assumindo a US-005 que seria de Allan.

Stack definida: Python 3.13 + FastAPI (back-end), HTML + Tailwind CSS (front-end), PostgreSQL 16 em nuvem (Render ou Supabase), API generativa (Gemini ou Claude) e Clean Architecture simplificada em quatro pacotes — web, core/service, core/persistencia e adapters. Repositório será hospedado no GitHub.

O MVP soma 19 Histórias de Usuário (US-001 a US-019), 128 pontos de história no total, distribuídas em 4 Sprints de 2 semanas (8 semanas) mais uma Sprint 0 de kickoff técnico.

1. Fluxo de Versionamento

1.1 Estrutura de Branches

Branch	Origem	Destino	Propósito	Proteção
main	—	—	Código em produção, sempre implantável. Deploy automático para Render/Vercel produção.	Protegida: só recebe merge de develop ou hotfix/* via PR aprovado + CI verde.
develop	main	main (fim de sprint)	Branch de integração contínua do time. Todo feature/fix nasce e volta para cá.	Protegida: exige PR + 1 aprovação + CI verde. Push direto bloqueado.

Branch	Origem	Destino	Propósito	Proteção
feature/US-XXX-slug	develop	develop	Uma branch por História de Usuário, um único dono.	—
fix/slug	develop	develop	Correção de bug encontrado durante a sprint.	—
release/sprintN	develop	main + develop	Opcional: estabilização de 1-2 dias ao fim de cada sprint antes de ir para main.	—
hotfix/slug	main	main + develop	Correção urgente em produção.	—

1.2 Convenção de Nomes de Branch

`<tipo>/US-<NNN>-<descricao-curta-em-kebab-case>`

Tipos aceitos: feature, fix, chore, docs, refactor, test.

Exemplos:

```
feature/US-001-cadastro-usuario
feature/US-006-analise-curriculo-ia
fix/US-004-validacao-tamanho-arquivo
chore/US-000-setup-projeto
```

Branches sem User Story associada (infraestrutura, setup) usam US-000.

1.3 Convenção de Commits (Conventional Commits)

`<tipo>(<escopo>): <descrição curta no imperativo>`

`<corpo opcional explicando o quê e o porquê>`

Refs: US-XXX

Tipos: feat, fix, docs, refactor, test, chore, style, perf. Escopo: nome do módulo afetado (auth, analise, diagnostico, simulador, templates, plano-dev, painel, infra).

Exemplos:

`feat(auth): implementa cadastro de usuário com validação de senha`

Adiciona endpoint POST /usuarios, hashing com bcrypt e envio de e-mail de confirmação.

Refs: US-001

`fix(analise): corrige rejeição de PDF válido acima de 4MB`

Refs: US-004

1.4 Estratégia Anti-Conflito (4 devs commitando em paralelo)

1. **Uma US = uma branch = um dono.** Nunca dois desenvolvedores commitando na mesma branch de feature.
2. **Divisão por módulo dentro de cada sprint** (ver Seção 3): cada dev trabalha preferencialmente em arquivos de um pacote (web/auth, core/service/analise, core/persistencia/vaga, etc.), reduzindo a chance de dois PRs tocarem o mesmo arquivo na mesma janela de tempo.
3. **Rebase diário obrigatório antes de abrir ou atualizar um PR:**

```
git checkout develop
git pull origin develop
git checkout feature/US-XXX-slug
git rebase develop
```

4. **PRs pequenos e frequentes.** Uma US grande (13 pts) deve ser quebrada em commits incrementais e, quando possível, em sub-PRs (ex.: primeiro o adapter, depois o service, depois a rota).
5. **Arquivos sensíveis (core/persistencia/models.py, alembic/versions/) exigem aviso prévio no canal do time antes de alterar o schema**, e o PR correspondente tem prioridade de revisão para evitar duas migrações divergentes simultâneas.
6. **Squash merge** no GitHub ao integrar em develop, mantendo o histórico linear e legível.
7. **CI obrigatório** (lint + testes) como status check antes de habilitar o botão de merge.

1.5 Regras de Proteção e Revisão

- main e develop: exigem Pull Request, mínimo 1 aprovação de outro desenvolvedor, e checks de CI (ruff + pytest) verdes.
 - Nenhum push direto a main ou develop, nem para administradores do repositório.
 - Uso de CODEOWNERS (Seção 2.5) para roteamento automático de revisores por módulo.
 - Conversas não resolvidas em um PR bloqueiam o merge (Require conversation resolution before merging).
-

2. Configuração Inicial

2.1 Estrutura de Pastas (Clean Architecture Simplificada)

Conforme o Documento de Arquitetura de Software (Seção 4, visão de implementação), o sistema é organizado em web/, core/service/, core/persistencia/ e adapters/, com fluxo de dependência unidirecional (web → core, core agnóstico a infraestrutura, adapters isolando serviços externos).

```
analizador-de-curriculos/
├── app/
│   ├── main.py
│   └── web/
│       ├── routers/
│       │   ├── auth_router.py
│       │   ├── analise_router.py
│       │   ├── diagnostico_router.py
│       │   ├── simulador_router.py
│       │   ├── templates_router.py
│       │   └── painel_router.py
```

```

    templates/
    static/
        css/
core/
    service/
        auth_service.py
        analisador_service.py
        diagnostico_service.py
        simulador_service.py
        template_service.py
        plano_service.py
    persistencia/
        models.py
        usuario_repository.py
        curriculo_repository.py
        vaga_repository.py
        analise_repository.py
    adapters/
        ai_service/
            ai_service_adapter.py
        curriculo_parser/
            curriculo_parser.py
alembic/
    versions/
tests/
    unit/
    integration/
.env.example
.gitignore
requirements.txt
Dockerfile
docker-compose.yml
CODEOWNERS
README.md

```

2.2 Passo a Passo — Setup Local

Criação do repositório e clone:

```

gh repo create analisador-de-curriculos --private --clone
cd analisador-de-curriculos

```

Criação da estrutura de pastas:

```

mkdir -p app/web/routers app/web/templates app/web/static/css
mkdir -p app/core/service app/core/persistencia
mkdir -p app/adapters/ai_service app/adapters/curriculo_parser
mkdir -p tests/unit tests/integration
mkdir -p alembic/versions

```

```

touch app/__init__.py app/web/__init__.py app/web/routers/__init__.py
touch app/core/__init__.py app/core/service/__init__.py app/core/persistencia/__init__.py
touch app/adapters/__init__.py app/adapters/ai_service/__init__.py app/adapters/curriculo_parser/__init__.py

```

Ambiente virtual e dependências:

```
python3.13 -m venv venv
source venv/bin/activate
```

```
pip install fastapi "uvicorn[standard]" sqlalchemy psycpg2-binary alembic \
    pydantic pydantic-settings python-dotenv python-multipart \
    jinja2 pymupdf python-docx "passlib[bcrypt]" "python-jose[cryptography]" \
    google-generativeai anthropic pytest pytest-asyncio httpx ruff
```

```
pip freeze > requirements.txt
```

Arquivo .gitignore:

```
venv/
__pycache__/
*.pyc
.env
.DS_Store
*.db
.pytest_cache/
.ruff_cache/
```

Primeiro commit e branches base:

```
git add .
git commit -m "chore(infra): estrutura inicial do projeto em Clean Architecture simplificada"
git branch develop
git push -u origin main
git push -u origin develop
```

Fluxo que cada desenvolvedor segue a partir daqui:

```
git clone https://github.com/<org>/analizador-de-curriculos.git
cd analisador-de-curriculos
git checkout develop
git checkout -b feature/US-001-cadastro-usuario
```

2.3 Docker e Banco de Dados Local

docker-compose.yml — Postgres local para desenvolvimento, espelhando a versão 16 usada em produção:

```
services:
  db:
    image: postgres:16
    restart: always
    environment:
      POSTGRES_USER: analisador
      POSTGRES_PASSWORD: analisador
      POSTGRES_DB: analisador_curriculos
    ports:
      - "5432:5432"
    volumes:
      - pgdata:/var/lib/postgresql/data
volumes:
  pgdata:
```

Dockerfile da aplicação:

```

FROM python:3.13-slim
WORKDIR /app
COPY requirements.txt .
RUN pip install --no-cache-dir -r requirements.txt
COPY . .
CMD ["uvicorn", "app.main:app", "--host", "0.0.0.0", "--port", "8000"]

.env.example:
DATABASE_URL=postgresql+psycopg2://analizador:analizador@localhost:5432/analizador_curriculos
SECRET_KEY=troque-por-uma-chave-secreta-forte
GEMINI_API_KEY=
ANTHROPIC_API_KEY=
ENVIRONMENT=development

```

2.4 Integração Contínua

```

.github/workflows/ci.yml:
name: CI
on:
  pull_request:
    branches: [develop, main]
jobs:
  test:
    runs-on: ubuntu-latest
    services:
      postgres:
        image: postgres:16
        env:
          POSTGRES_USER: test
          POSTGRES_PASSWORD: test
          POSTGRES_DB: test_db
        ports: ["5432:5432"]
        options: >-
          --health-cmd pg_isready
          --health-interval 10s
          --health-timeout 5s
          --health-retries 5
    steps:
      - uses: actions/checkout@v4
      - uses: actions/setup-python@v5
        with:
          python-version: "3.13"
      - run: pip install -r requirements.txt
      - run: ruff check .
      - run: pytest

```

2.5 CODEOWNERS e Proteção de Branches

CODEOWNERS (ajustar @usuario-github para o handle real de cada dev):

```

/app/web/                @kevin-gh @gabrielkalebe-gh
/app/core/service/       @carloseduardo-gh @gabrielkalebe-gh
/app/core/persistencia/  @allandasilva-gh @kevin-gh

```

```
/app/adapters/          @gustavosouto-gh @carloseduardo-gh
/alembic/                @allandasilva-gh @kevin-gh
```

Habilitação das regras de proteção via GitHub CLI (executar com permissão de admin no repositório):

```
gh api repos/:owner/:repo/branches/main/protection -X PUT \
-f required_status_checks[strict]=true \
-f required_status_checks[contexts][]=test \
-f enforce_admins=true \
-f required_pull_request_reviews[required_approving_review_count]=1 \
-f restrictions=null
```

```
gh api repos/:owner/:repo/branches/develop/protection -X PUT \
-f required_status_checks[strict]=true \
-f required_status_checks[contexts][]=test \
-f enforce_admins=true \
-f required_pull_request_reviews[required_approving_review_count]=1 \
-f restrictions=null
```

Adição dos colaboradores:

```
gh repo add-collaborator gabrielkalebe-gh --permission push
gh repo add-collaborator carloseduardo-gh --permission push
gh repo add-collaborator allandasilva-gh --permission push
gh repo add-collaborator gustavosouto-gh --permission push
```

3. Divisão de Tarefas — Sprints

Equipe considerada: **Kevin Carvalho Iqbal (K)**, **Gabriel Kalebe (G)**, **Carlos Eduardo Oliveira (C)**, **Allan da Silva (A)**, **Gustavo Souto Pereira (Gu)** e **Vitor Bittencourt dos Santos (V)**. Gustavo entra já na Sprint 1 assumindo a fatia que seria de Carlos (US-004 + apoio de infraestrutura); Carlos passa a atuar a partir da Sprint 2, mantendo o que já estava previsto para ele nas Sprints 2 a 4. Vitor também entra já na Sprint 1, assumindo a US-005 que seria de Allan; Allan segue normalmente com US-017 nesta sprint e com o restante do que já estava previsto para ele. A alocação de Vitor a partir da Sprint 2 ainda não está fixada — o time deve defini-la ao final da Sprint 1, com um módulo completo (ex.: Painel + Biblioteca) como ponto de entrada natural.

3.1 Visão Geral

Sprint	Semanas	Foco	Pontos
Sprint 0	Pré-Sprint 1 (2-3 dias)	Kickoff técnico: repositório, estrutura, CI/CD, Docker, deploy esqueleto	—
Sprint 1	1-2	Autenticação, upload de currículo, ingestão de vaga	29 pts

Sprint	Semanas	Foco	Pontos
Sprint 2	3-4	Núcleo de IA: análise de currículo, comparação com vaga, diagnóstico	42 pts
Sprint 3	5-6	Simulador de entrevistas e templates ATS	34 pts
Sprint 4	7-8	Plano de desenvolvimento, biblioteca, painel, hardening e deploy final	23 pts

Total: 128 pontos, distribuídos de forma a garantir que cada desenvolvedor passe, ao longo do projeto, por trabalho de autenticação/banco, por integração com a API de IA e por construção de telas — nenhum dev fica restrito a um único tipo de tarefa.

3.2 Sprint 0 — Kickoff Técnico (antes da Sprint 1)

Trabalho conjunto do time inteiro, não distribuído por dono único:

- Criação do repositório GitHub, branches `main/develop`, proteção de branch, CODEOWNERS.
- Estrutura de pastas conforme Seção 2.1.
- `docker-compose.yml` com Postgres 16 local.
- Configuração do FastAPI mínimo (`app/main.py` com rota de health-check) e template Jinja2 + Tailwind via CDN ou build simples.
- Pipeline de CI (lint + testes) no GitHub Actions.
- Criação do banco gerenciado (Render Postgres ou Supabase) e do serviço web no Render, apontando para `main` (deploy inicial vazio, só para validar o pipeline de implantação).
- Definição das chaves de API (Gemini/Claude) como variáveis de ambiente seguras no provedor de nuvem.

3.3 Sprint 1 (Semanas 1-2) — Fundação: Autenticação e Ingestão

Dev	US	Título	Pontos	Módulo principal
K	US-001	Cadastrar conta de usuário	5	web/auth, core/service/auth_service, core/persistencia/usuario_repo
K	US-002	Realizar login com e-mail e senha	3	web/auth, core/service/auth_service
G	US-003	Recuperar senha via e-mail	3	core/service/auth_service
G	US-019	Cadastrar informações de vaga no banco	5	core/persistencia/vaga_reposit
Gu	US-004	Fazer upload do currículo (PDF/DOCX)	5	adapters/curriculo_parser, web/analise

Dev	US	Título	Pontos	Módulo principal
Gu	—	Apoio técnico: Docker, CI, Alembic (migração inicial)	—	Infraestrutura
V	US-005	Colar descrição textual de uma vaga	3	web/analise
A	US-017	Excluir conta e dados (LGPD)	5	core/service/auth_service, core/persistencia

Dependências desta sprint: US-002/US-003/US-017 dependem de US-001; US-004 e US-005 dependem de US-002; US-019 é independente e pode começar em paralelo. Recomenda-se que K entregue US-001 nos primeiros 2-3 dias para destravar G, Gu, A e V.

Carlos Eduardo Oliveira não tem entrega nesta sprint — ele volta a aparecer na Sprint 2 com o que já estava planejado para ele (US-006, o núcleo de análise por IA), sem nenhuma outra mudança no restante do plano.

3.4 Sprint 2 (Semanas 3-4) — Núcleo de IA: Análise e Diagnóstico

Dev	US	Título	Pontos	Módulo principal
C	US-006	Analisar conteúdo do currículo (IA)	13	adapters/ai_service, core/service/analizador_servi
K	US-007	Comparar currículo com vaga e ver aderência	13	core/service/analizador_servi (Fuzzy Matching/Tokenização)
G	US-008	Receber diagnóstico crítico do currículo	8	core/service/diagnostico_servi
A	US-009	Obter sugestões de reescrita de trechos fracos	8	core/service/diagnostico_servi

Esta é a sprint mais pesada (42 pts) por concentrar o core de IA do produto. US-008 depende da entrega de US-006 (C) e US-009 depende de US-008 (G); recomenda-se pareamento pontual entre C+G e G+A nos primeiros dias para não travar a esteira. C e K são os dois devs com contato direto à API generativa nesta sprint — o AIServiceAdapter (Seção 4.2/4.3 da Arquitetura) deve ser construído por C já pensando em ser consumido também por US-010 na Sprint 3.

3.5 Sprint 3 (Semanas 5-6) — Simulador de Entrevistas e Templates ATS

Dev	US	Título	Pontos	Módulo principal
A	US-012	Acessar templates de currículo	5	web/templates, core/service/template_service
C	US-013	ATS-friendly Aplicar currículo a template e exportar PDF/DOCX	8	core/service/template_service
K	US-010	Simular entrevista com perguntas por vaga (IA)	13	adapters/ai_service, core/service/simulador_service
G	US-011	Receber feedback da resposta de entrevista	8	core/service/simulador_service

US-013 depende da entrega de US-012 (A); US-011 depende de US-010 (K). Nesta sprint C e A assumem o fluxo de telas/exportação (equilibrando o trabalho mais “backend/IA” que tiveram nas sprints anteriores), enquanto K leva sua segunda US de integração direta com IA e G ganha experiência processando saída de IA em US-011.

3.6 Sprint 4 (Semanas 7-8) — Plano de Desenvolvimento, Biblioteca, Painel e Hardening

Dev	US	Título	Pontos	Módulo principal
K	US-014	Receber recomendações para evidenciar competências	5	core/service/plano_service
G	US-015	Visualizar plano de desenvolvimento e certificações sugeridas	5	core/service/plano_service
C	US-016	Acessar painel com histórico e evolução das análises	8	web/painel, core/persistencia/analise_repo
A	US-018	Acessar biblioteca de currículos	5	web/painel, core/persistencia/curriculo_re

US-015 depende de US-014 (K); US-016 e US-018 dependem de US-007/US-004/US-013 já entregues nas sprints anteriores. Sprint intencionalmente mais leve (23 pts) para reservar a segunda semana a: testes de carga simulando RNF-001 (30s de resposta) e RNF-007 (100 usuários simultâneos), checklist de conformidade LGPD (RNF-003), deploy final em produção (Render/Supabase + Vercel), revisão de segurança e ensaio da apresentação/demo do MVP.

3.7 Observações de Balanceamento

- Cada desenvolvedor passa por pelo menos uma US de autenticação/dados (Sprint 1), uma US com integração direta à API de IA (K: Sprint 2 e 3; C: Sprint 2; G e A: consumo de resultado de IA em US-008/009/011) e uma US de tela/exportação (Sprint 1 ou 3). Gustavo, entrando pela Sprint 1, já começa pelo módulo de ingestão (upload + parser), o mesmo ponto de entrada que Carlos teria — a rotação prevista para Carlos nas Sprints 2 a 4 continua valendo normalmente.
 - K e C concentram as duas US de 13 pontos que envolvem chamada direta ao adapter de IA (US-006, US-007, US-010); isso é proposital para dar continuidade técnica ao `AIServiceAdapter`, mas o pareamento sugerido nas Seções 3.4 e 3.5 garante que G e A também acompanhem essa camada de perto.
 - Vitor Bittencourt dos Santos entrou direto na Sprint 1 com a US-005, que seria de Allan; Allan segue normalmente com US-017 e com o restante do que já estava previsto para ele nas próximas sprints — nenhuma outra mudança no plano dele. A alocação de Vitor a partir da Sprint 2 fica em aberto; sugerimos que assuma um módulo completo (ex.: Painel + Biblioteca) a partir da Sprint 3 ou 4 e, no ciclo pós-MVP, os itens que hoje estão fora do escopo das 19 US (ex.: novos perfis de usuário, idiomas, certificações — citados na Seção 6.5 da Arquitetura como extensões futuras do modelo de dados).
-

4. Próximos Passos

1. Confirmar provedor de e-mail transacional para US-001/US-003 (ex.: SendGrid, Resend) — não definido nos documentos de origem.
2. Confirmar entre Gemini API e Anthropic Claude API como provedor principal do `AIServiceAdapter` (a Arquitetura lista ambos como opção).
3. Criar quadro no GitHub Projects espelhando as tabelas das Seções 3.3 a 3.6, com uma coluna por status (A Fazer, Em Progresso, Em Revisão, Concluído).
4. Definir horário fixo de daily (15 min) e formato de sprint review/retro ao fim de cada sprint de 2 semanas.
5. Registrar, junto à equipe, os handles reais do GitHub de cada desenvolvedor para popular o CODEOWNERS e os comandos `gh repo add-collaborator` da Seção 2.5.